ICS1503- Web Application Development

Exercise 8

Shopping Cart Full Stack Web Application using Vue.js, Spring Boot and MongoDB

Name : Balakumaran S
Class : MTech CSE (Integrated 5 years)
Reg No : 3122247001010 Date: 14/09/2026

Exercise Description

A Shopping Cart application allows users to view products, add products to a cart, update quantities, and remove products from the cart. In this exercise, a full stack application was developed using Vue.js for the frontend, Spring Boot for the backend, and MongoDB for data storage. The frontend sends requests to the backend REST API, and the backend reads and writes the data in MongoDB.

Frontend Layer – Vue.js

- Display the available products.
- Add products to the shopping cart.
- Update or remove products in the cart.
- Send requests to the backend.
- Display the cart details and the total amount.

Backend Layer – Spring Boot

- Process requests from the frontend.
- Add, update, and remove cart items.
- Calculate the total cart amount.
- Communicate with the database.

Database Layer – MongoDB

- Stores the application data in two main collections: Products and Cart.

Code

Backend (Spring Boot)

Product.java (Model)

```
package com.example.shoppingcart.model;

import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;
import org.springframework.data.annotation.Id;
import org.springframework.data.mongodb.core.mapping.Document;

@Data
@NoArgsConstructor
@AllArgsConstructor
@Document(collection = "products")
public class Product {

    @Id
    private String id;

    private String name;

    private String description;

    private double price;
}
```

CartItem.java (Model)

```
package com.example.shoppingcart.model;

import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;
import org.springframework.data.annotation.Id;
import org.springframework.data.mongodb.core.mapping.Document;

@Data
@NoArgsConstructor
@AllArgsConstructor
@Document(collection = "cart")
public class CartItem {

    @Id
    private String id;

    private String productId;
```

```
    private String name;

    private double price;

    private int quantity;
}
```

AddToCartRequest.java (DTO)

```
package com.example.shoppingcart.dto;

import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@NoArgsConstructor
@AllArgsConstructor
public class AddToCartRequest {

    private String productId;

    private int quantity;
}
```

UpdateCartRequest.java (DTO)

```
package com.example.shoppingcart.dto;

import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@NoArgsConstructor
@AllArgsConstructor
public class UpdateCartRequest {

    private String productId;

    private int quantity;
}
```

ProductRepository.java (Repository)

```
package com.example.shoppingcart.repository;

import com.example.shoppingcart.model.Product;
import org.springframework.data.mongodb.repository.MongoRepository;
```

```
public interface ProductRepository extends MongoRepository<Product, String> {
}
```

CartRepository.java (Repository)

```
package com.example.shoppingcart.repository;

import com.example.shoppingcart.model.CartItem;
import org.springframework.data.mongodb.repository.MongoRepository;

import java.util.Optional;

public interface CartRepository extends MongoRepository<CartItem, String> {

    Optional<CartItem> findByProductId(String productId);

    void deleteByProductId(String productId);
}
```

ProductService.java (Service)

```
package com.example.shoppingcart.service;

import com.example.shoppingcart.model.Product;
import com.example.shoppingcart.repository.ProductRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;

import java.util.List;
import java.util.Optional;

@Service
@RequiredArgsConstructor
public class ProductService {

    private final ProductRepository productRepository;

    // CREATE
    public Product addProduct(Product product) {
        return productRepository.save(product);
    }

    // READ - Get all products
    public List<Product> getAllProducts () {
        return productRepository.findAll();
    }

    // READ - Get product by ID
    public Optional<Product> getProductById(String id) {
        return productRepository.findById(id);
    }
```

```
    // UPDATE
    public Optional<Product> updateProduct(String id, Product updatedProduct) {

        return productRepository.findById(id)
                .map(existingProduct -> {

                    existingProduct.setName(updatedProduct.getName());

existingProduct.setDescription(updatedProduct.getDescription());
                    existingProduct.setPrice(updatedProduct.getPrice());

                    return productRepository.save(existingProduct);
                });
    }

    // DELETE
    public boolean deleteProduct(String id) {

        if (productRepository.existsById(id)) {
            productRepository.deleteById(id);
            return true;
        }

        return false;
    }
}
```

CartService.java (Service)

```
package com.example.shoppingcart.service;

import com.example.shoppingcart.dto.AddToCartRequest;
import com.example.shoppingcart.dto.UpdateCartRequest;
import com.example.shoppingcart.model.CartItem;
import com.example.shoppingcart.model.Product;
import com.example.shoppingcart.repository.CartRepository;
import com.example.shoppingcart.repository.ProductRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;

import java.util.List;

@Service
@RequiredArgsConstructor
public class CartService {

    private final CartRepository cartRepository;
    private final ProductRepository productRepository;

    public List<CartItem> getCart() {
        return cartRepository.findAll();
    }
```

```
    public CartItem addToCart(AddToCartRequest request) {

        Product product = productRepository.findById(request.getProductId())
                .orElseThrow(() -> new RuntimeException("Product not found"));

        CartItem existingItem = cartRepository
                .findByProductId(request.getProductId())
                .orElse(null);

        if (existingItem != null) {

            existingItem.setQuantity(
                    existingItem.getQuantity() + request.getQuantity()
            );

            return cartRepository.save(existingItem);
        }

        CartItem cartItem = new CartItem();

        cartItem.setProductId(product.getId());
        cartItem.setName(product.getName());
        cartItem.setPrice(product.getPrice());
        cartItem.setQuantity(request.getQuantity());

        return cartRepository.save(cartItem);
    }

    public CartItem updateQuantity(UpdateCartRequest request) {

        CartItem cartItem = cartRepository
                .findByProductId(request.getProductId())
                .orElseThrow(() -> new RuntimeException("Cart item not found"));

        if (request.getQuantity() < 1) {
            throw new RuntimeException("Quantity must be at least 1");
        }

        cartItem.setQuantity(request.getQuantity());

        return cartRepository.save(cartItem);
    }

    public void removeFromCart(String productId) {

        CartItem cartItem = cartRepository
                .findByProductId(productId)
                .orElseThrow(() -> new RuntimeException("Cart item not found"));

        cartRepository.delete(cartItem);
    }

    public double getTotalAmount() {
```

```
        return cartRepository.findAll()
                .stream()
                .mapToDouble(item -> item.getPrice() * item.getQuantity())
                .sum();
    }
}
```

ProductController.java (Controller)

```
package com.example.shoppingcart.controller;

import com.example.shoppingcart.model.Product;
import com.example.shoppingcart.service.ProductService;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/products")
@CrossOrigin(origins = "http://localhost:5173")
@RequiredArgsConstructor
public class ProductController {

    private final ProductService productService;

    // CREATE
    @PostMapping
    public ResponseEntity<Product> addProduct(@RequestBody Product product) {

        Product savedProduct = productService.addProduct(product);

        return new ResponseEntity<>(savedProduct, HttpStatus.CREATED);
    }

    // READ - Get all products
    @GetMapping
    public List<Product> getAllProducts() {
        return productService.getAllProducts();
    }

    // READ - Get product by ID
    @GetMapping("/{id}")
    public ResponseEntity<Product> getProductById(@PathVariable String id) {

        return productService.getProductById(id)
                .map(ResponseEntity::ok)
                .orElse(ResponseEntity.notFound().build());
    }

    // UPDATE
```

```
    @PutMapping("/{id}")
    public ResponseEntity<Product> updateProduct(
            @PathVariable String id,
            @RequestBody Product product) {

        return productService.updateProduct(id, product)
                .map(ResponseEntity::ok)
                .orElse(ResponseEntity.notFound().build());
    }

    // DELETE
    @DeleteMapping("/{id}")
    public ResponseEntity<Void> deleteProduct(@PathVariable String id) {

        boolean deleted = productService.deleteProduct(id);

        if (deleted) {
            return ResponseEntity.noContent().build();
        }

        return ResponseEntity.notFound().build();
    }
}
```

CartController.java (Controller)

```
package com.example.shoppingcart.controller;

import com.example.shoppingcart.dto.AddToCartRequest;
import com.example.shoppingcart.dto.UpdateCartRequest;
import com.example.shoppingcart.model.CartItem;
import com.example.shoppingcart.service.CartService;
import lombok.RequiredArgsConstructor;
import org.springframework.web.bind.annotation.*;

import java.util.List;
import java.util.Map;

@RestController
@RequestMapping("/api/cart")
@CrossOrigin(origins = "http://localhost:5173")
@RequiredArgsConstructor
public class CartController {

    private final CartService cartService;

    @GetMapping
    public List<CartItem> getCart() {
        return cartService.getCart();
    }

    @PostMapping("/add")
    public CartItem addToCart(@RequestBody AddToCartRequest request) {
```

```
        return cartService.addToCart(request);
    }

    @PutMapping("/update")
    public CartItem updateQuantity(
            @RequestBody UpdateCartRequest request) {
        return cartService.updateQuantity(request);
    }

    @DeleteMapping("/remove/{productId}")
    public void removeFromCart(@PathVariable String productId) {
        cartService.removeFromCart(productId);
    }

    @GetMapping("/total")
    public Map<String, Double> getTotalAmount() {
        return Map.of(
                "total", cartService.getTotalAmount()
        );
    }
}
```

ShoppingCartApplication.java

```
package com.example.shoppingcart;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class ShoppingCartApplication {

    public static void main(String[] args) {
        SpringApplication.run(ShoppingCartApplication.class, args);
    }

}
```

application.properties

```
spring.application.name=ShoppingCart

# Replace host/port/database lines with spring.data.mongodb.uri
spring.mongodb.uri=mongodb://localhost:27017/shopping_cart

server.port=8080
logging.level.org.mongodb.driver.protocol.command=DEBUG
```

Frontend (Vue.js)

main.js

```
import { createApp } from 'vue'
import App from './App.vue'
import router from './router'

createApp(App).use(router).mount('#app')
```

router/index.js

```
import {createRouter, createWebHistory} from 'vue-router'
import Products from '../views/Products.vue'
import Cart from '../views/Cart.vue'

const routes = [
    {
        path: '/',
        redirect: '/products'
    },
    {
        path: '/products',
        component: Products
    },
    {
        path: '/cart',
        component: Cart
    }
]

const router = createRouter({
    history: createWebHistory(),
    routes
})

export default router
```

App.vue

```
<script setup>
import Navbar from './components/Navbar.vue';
</script>

<template>
  <Navbar />

  <main>
    <router-view/>
  </main>
```

```
</template>

<style scoped></style>
```

services/api.js

```
import axios from 'axios'

const api = axios.create({
  baseURL: 'http://localhost:8080/api'
})

export default api
```

components/Navbar.vue

```
<script setup></script>

<template>
  <nav class="navbar">
    <div class="logo">
      Shopping Cart
    </div>

    <div class="nav-links">
      <router-link to="/products">
        Products
      </router-link>

      <router-link to="/cart">
        Cart
      </router-link>
    </div>
  </nav>
</template>

<style scoped>
.navbar {
  height: 70px;
  padding: 0 40px;
  display: flex;
  align-items: center;
  justify-content: space-between;
  border-bottom: 1px solid #ddd;
  background-color: white;
}

.logo {
  font-size: 22px;
  font-weight: 600;
```

```
}

.nav-links {
  display: flex;
  gap: 25px;
  cursor: pointer;
}

.nav-links a {
  text-decoration: none;
  color: #333;
  font-size: 16px;

}

.nav-links a.router-link-active {
  font-weight: 600;
}
</style>
```

components/ProductCard.vue

```
<script setup>
defineProps({
  product: {
    type: Object,
    required: true
  }
})

const emit = defineEmits(['add-to-cart'])
</script>

<template>
  <div class="product-card">
    <div class="product-image">
      <span>{{ product.name.charAt(0) }}</span>
    </div>

    <div class="product-details">
      <h2>{{ product.name }}</h2>

      <p class="description">
        {{ product.description }}
      </p>

      <p class="price">
        ₹{{ product.price }}
      </p>

      <button @click="emit('add-to-cart', product)">
        Add to Cart
      </button>
```

```
    </div>
  </div>
</template>

<style scoped>
.product-card {
  border: 1px solid #ddd;
  border-radius: 10px;
  overflow: hidden;
  background-color: white;
}

.product-image {
  height: 180px;
  display: flex;
  align-items: center;
  justify-content: center;
  background-color: #f3f3f3;
}

.product-image span {
  font-size: 50px;
  font-weight: 600;
  color: #777;
}

.product-details {
  padding: 20px;
}

.product-details h2 {
  margin: 0 0 10px;
  font-size: 20px;
}

.description {
  color: #666;
  min-height: 40px;
}

.price {
  font-size: 18px;
  font-weight: 600;
  margin: 15px 0;
}

button {
  width: 100%;
  padding: 10px;
  border: none;
  border-radius: 6px;
  cursor: pointer;
}
```

```
button:hover {
  background-color: black;
  color:white;
}

</style>
```

components/CartItem.vue

```
<script setup>
defineProps({
  item: {
    type: Object,
    required: true
  }
})

const emit = defineEmits(['increase', 'decrease', 'remove'])
</script>

<template>

  <div class="cart-item">
    <div class="item-details">
      <h3>{{ item.name }}</h3>
      <p>₹{{ item.price }} each</p>
    </div>

    <div class="quantity">
      <button @click="emit('decrease')">
        -
      </button>

      <span>{{ item.quantity }}</span>

      <button @click="emit('increase')">
        +
      </button>
    </div>

    <div class="item-total">
      ₹{{ item.price * item.quantity }}
    </div>

    <button class="remove-button" @click="emit('remove')">
      Remove
    </button>
  </div>
</template>

<style scoped>
.cart-item {
  display: flex;
```

```
  align-items: center;
  gap: 30px;
  padding: 20px;
  border-bottom: 1px solid #ddd;
}

.item-details {
  flex: 1;
}

.item-details h3 {
  margin: 0 0 5px;
}

.item-details p {
  margin: 0;
  color: #666;
}

.quantity {
  display: flex;
  align-items: center;
  gap: 15px;
}

.quantity button {
  width: 30px;
  height: 30px;
  border: 1px solid #ccc;
  background-color: white;
  cursor: pointer;
}

.item-total {
  font-weight: 600;
}

.remove-button {
  border: none;
  background-color: transparent;
  cursor: pointer;
}
</style>
```

views/Products.vue

```
<script setup>
import { onMounted, ref } from 'vue'
import ProductCard from '../components/ProductCard.vue'
import api from '../services/api.js'

const products = ref([])
const loading = ref(true)
```

```
const error = ref('')

const fetchProducts = async () => {
  try {
    const response = await api.get('/products')
    products.value = response.data
  } catch (err) {
    error.value = 'Unable to load products.'
  } finally {
    loading.value = false
  }
}

const addToCart = async (product) => {
  try {
    await api.post('/cart/add', {
      productId: product.id,
      quantity: 1
    })
    alert("Added to cart!")
  } catch (err) {
    error.value = 'Unable to add product to cart.'
  }
}

onMounted(fetchProducts)
</script>

<template>

  <div class="products-page">
    <h1>Products</h1>

    <p v-if="loading">
      Loading products...
    </p>

    <p v-else-if="error">
      {{ error }}
    </p>

    <div v-else class="product-grid">
      <ProductCard
        v-for="product in products"
        :key="product.id"
        :product="product"
        @add-to-cart="addToCart"
      />
    </div>
  </div>
</template>

<style scoped>
.products-page {
```

```
  max-width: 1200px;
  margin: 0 auto;
  padding: 40px 20px;
}

.products-page h1 {
  margin-bottom: 30px;
}

.product-grid {
  display: grid;
  grid-template-columns: repeat(4, 1fr);
  gap: 25px;
}

@media (max-width: 900px) {
  .product-grid {
    grid-template-columns: repeat(2, 1fr);
  }
}

@media (max-width: 600px) {
  .product-grid {
    grid-template-columns: 1fr;
  }
}
</style>
```

views/Cart.vue

```
<script setup>
import { computed, onMounted, ref } from 'vue'
import CartItem from '../components/CartItem.vue'
import api from '../services/api'

const cartItems = ref([])
const loading = ref(true)
const error = ref('')

const fetchCart = async () => {
  try {
    const response = await api.get('/cart')
    cartItems.value = response.data
  } catch (err) {
    error.value = 'Unable to load cart.'
  } finally {
    loading.value = false
  }
}

const placeOrder = async () => {
  try {
    const response = await api.post('/orders')
```

```
    alert('Order placed successfully!')

    cartItems.value = []

  } catch (err) {
    alert('Unable to place order.')
  }
}

const updateQuantity = async (item, quantity) => {
  if (quantity < 1) {
    return
  }

  try {
    await api.put('/cart/update', {
      productId: item.productId,
      quantity: quantity
    })

    await fetchCart()
  } catch (err) {
    error.value = 'Unable to update cart.'
  }
}

const removeItem = async (item) => {
  try {
    await api.delete(`/cart/remove/${item.productId}`)
    await fetchCart()
  } catch (err) {
    error.value = 'Unable to remove item.'
  }
}

const totalAmount = computed(() => {
  return cartItems.value.reduce(
    (total, item) => total + item.price * item.quantity,
    0
  )
})

onMounted(fetchCart)
</script>

<template>

  <div class="cart-page">
    <h1>Shopping Cart</h1>

    <p v-if="loading">
      Loading cart...
    </p>
```

```
    <p v-else-if="error">
      {{ error }}
    </p>

    <p v-else-if="cartItems.length === 0">
      Your cart is empty.
    </p>

    <template v-else>
      <div class="cart-list">
        <CartItem
          v-for="item in cartItems"
          :key="item.productId"
          :item="item"
          @increase="updateQuantity(item, item.quantity + 1)"
          @decrease="updateQuantity(item, item.quantity - 1)"
          @remove="removeItem(item)"
        />
      </div>

      <div class="cart-total">
        <h2>Total: ₹{{ totalAmount }}</h2>
      </div>

      <button
        v-if="cartItems.length > 0"
        @click="placeOrder"
      >
        Buy
      </button>

    </template>
  </div>
</template>

<style scoped>
.cart-page {
  max-width: 1000px;
  margin: 0 auto;
  padding: 40px 20px;
}

.cart-page h1 {
  margin-bottom: 30px;
}

.cart-list {
  border: 1px solid #ddd;
  border-radius: 10px;
  overflow: hidden;
}

.cart-total {
```

```
  display: flex;
  justify-content: flex-end;
  padding: 25px 0;
}
</style>
```

Output Snapshot

Backend Running

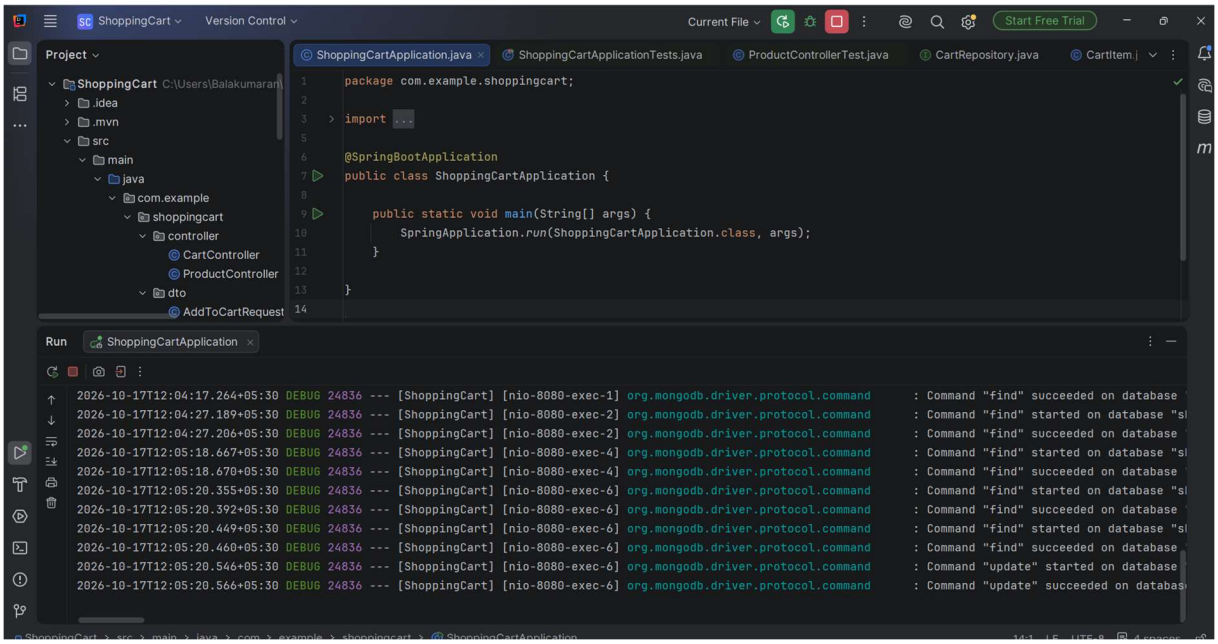


Fig 1: Spring Boot backend running and executing MongoDB commands

MongoDB - Products Collection

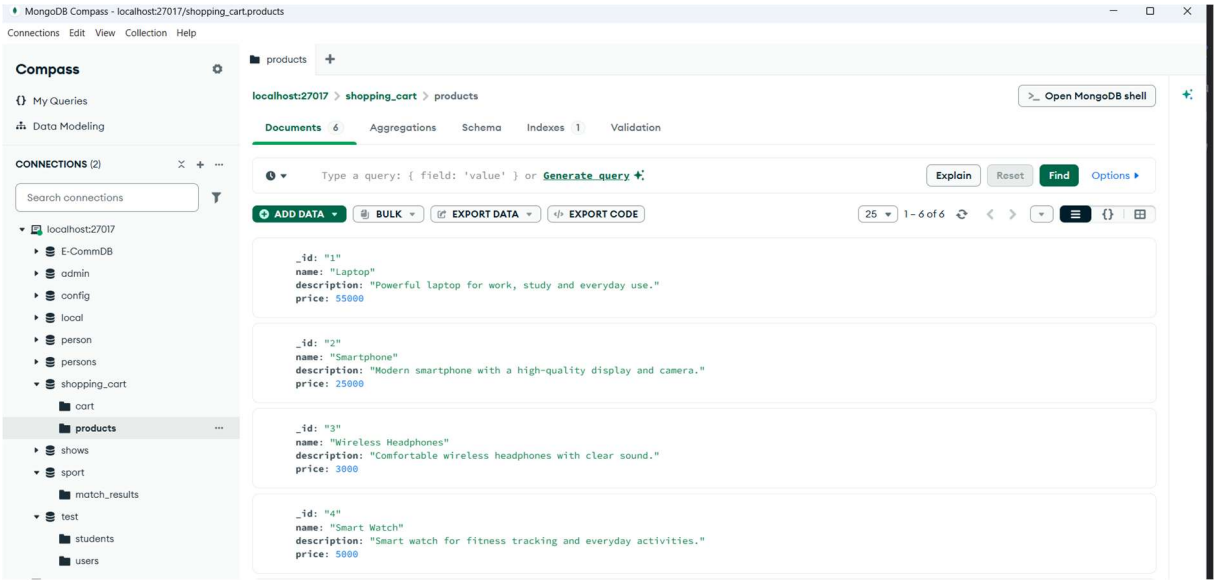


Fig 2: Products collection in the shopping_cart database

Products Page

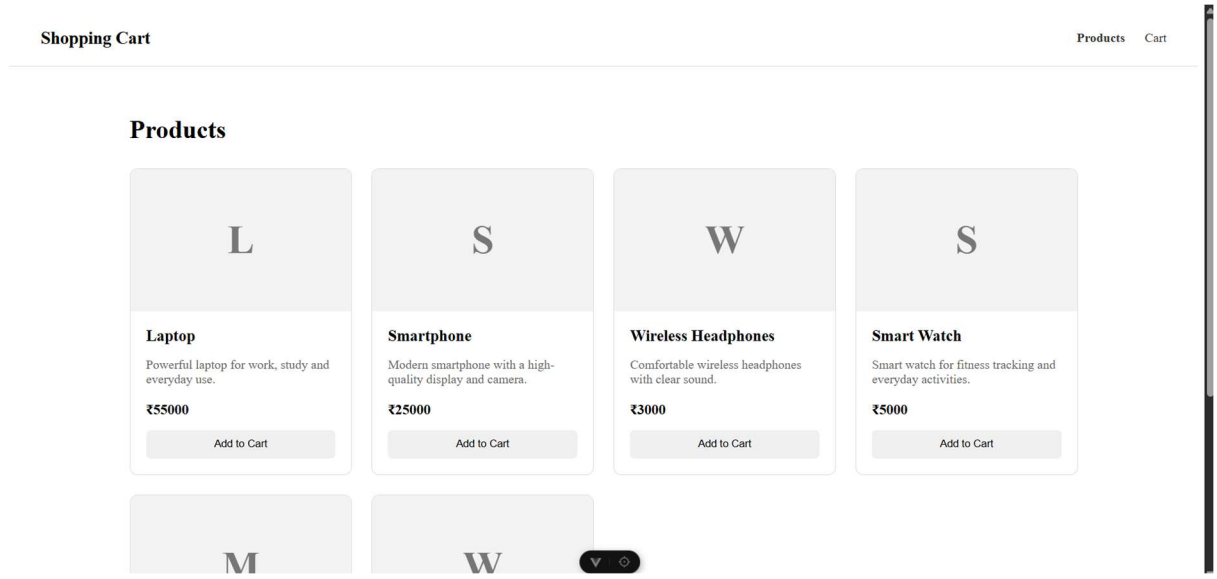


Fig 3: Products fetched from the backend and displayed on the Vue.js frontend

Add to Cart

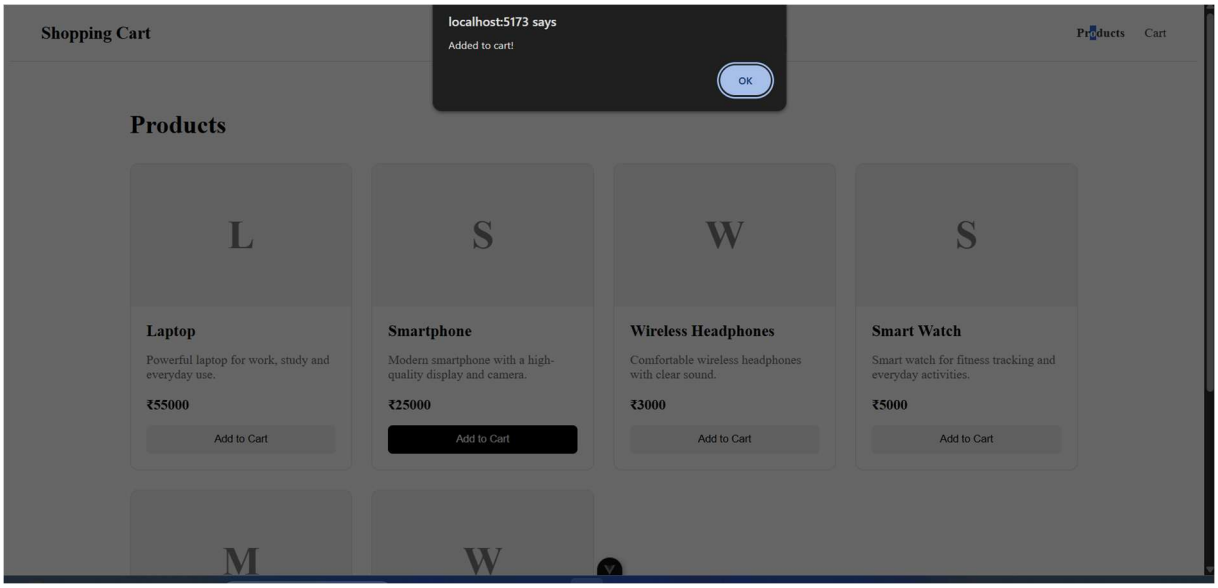


Fig 4: Product added to the cart, with the confirmation message

Cart Page

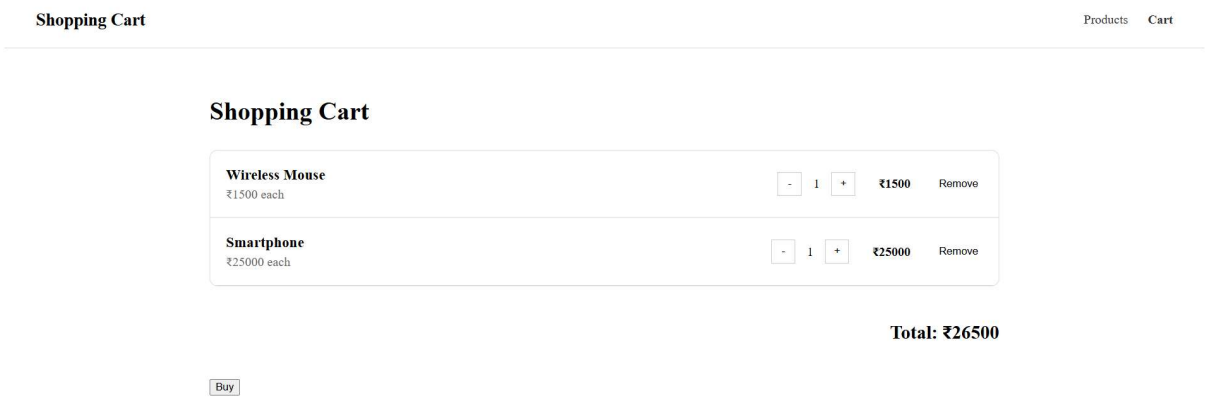


Fig 5: Cart page showing the items, quantity controls, remove option and total amount

MongoDB - Cart Collection

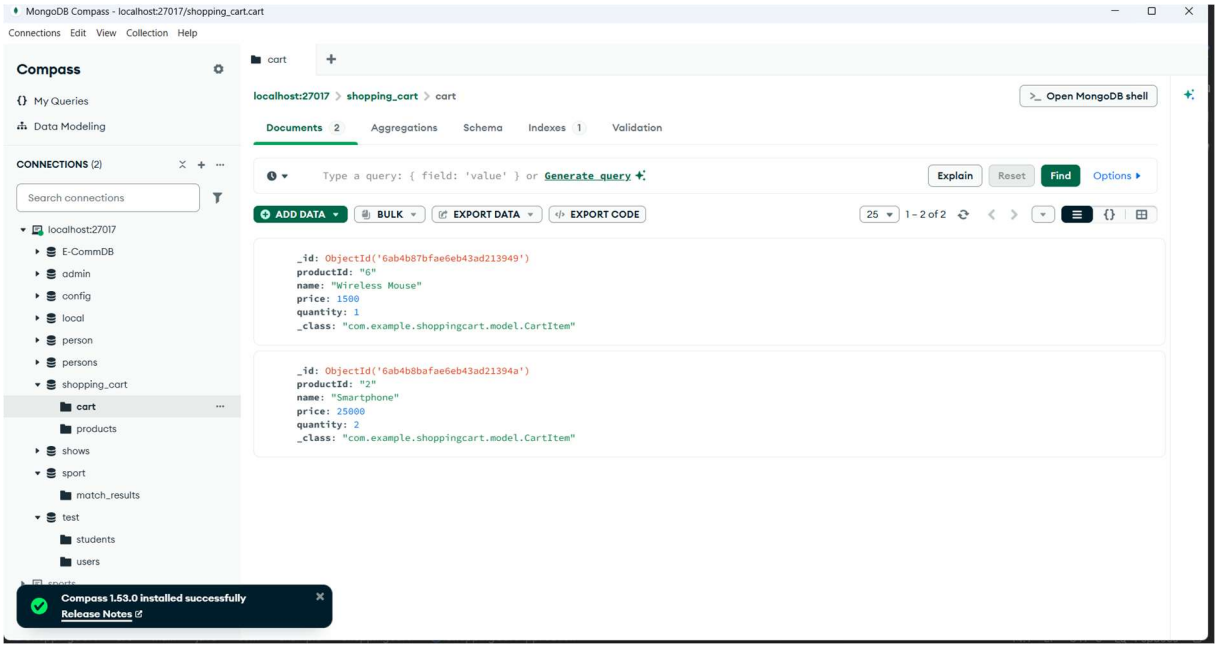


Fig 6: Cart collection in MongoDB storing the cart items with product ID, price and quantity

Learning Outcome

- Understood how a full stack web application is divided into frontend, backend, and database layers, and how they communicate.
- Learned how to build a REST API in Spring Boot with controller, service, and repository layers for products and cart operations.
- Learned how Spring Data MongoDB stores Java objects as documents in the Products and Cart collections.
- Practiced building a Vue.js frontend with routing, reusable components, and views for the Products and Cart pages.
- Learned how to send requests from Vue.js to the backend using axios, and how to enable CORS so the two applications can communicate.
- Understood how cart operations such as adding, updating quantity, removing items, and calculating the total amount work from the UI down to the database.